Practicum Test - Example

Instructions

- **Test Duration**: 25 minutes
- Ensure all function names match the specification exactly.
- No external resources or AI assistance permitted.

Question 1

WHAT YOU ARE ASKED

NOTE:

- You are required to write the solutions for the parts of this question in the lisp program file **~/quicklisp/local-projects/5ig/Example/pt/q1.lisp** .
- You may create helper functions in your program file.
- To ensure your solution is in the correct folder and passes the test cases shown in the examples below, type the following expression on the REPL:

```
(cg:chk-my-solution "~/quicklisp/local-projects/5ig/Example/pt/q1.lisp")
```

Write a Common Lisp function called DOT-PRODUCT-LIST that takes two lists of numbers of equal length. The function should return a list where each element is the product of the numbers at that position.

In your implementation, you must write and use a helper function called MULTIPLY-RECORDS that takes two numbers and returns their product. You may implement this as a standalone function or locally using LABELS.

```
The expression below

  (DOT-PRODUCT-LIST '(1 2 3) '(4 5 6))

should evaluate to

  (4 10 18)

The expression below

  (DOT-PRODUCT-LIST NIL NIL)

should evaluate to

  NIL
```

Question 2

WHAT YOU ARE ASKED

NOTE:

- You are required to write the solutions for the parts of this question in the lisp program file ~/**quicklisp/local-projects/5ig/Example/pt/q2.lisp** .
- You may create helper functions in your program file.
- To ensure your solution is in the correct folder and passes the test cases shown in the examples below, type the following expression on the REPL:

```
(cg:chk-my-solution "~/quicklisp/local-projects/5ig/Example/pt/q2.lisp")
```

Write a function SQUARE-EVENS that takes a list of integers and returns a new list containing the squares of only the even numbers in the original list, maintaining their relative order.

Constraint: You are forbidden from using REVERSE or APPEND. You must build the list in the correct order using recursion or iterative collection.

```
The expression below
```

```
  (SQUARE-EVENS '(1 2 3 4 5 6))
```

should evaluate to

```
  (4 16 36)
```

The expression below

```
  (SQUARE-EVENS '(1 3 5))
```

should evaluate to

```
  NIL
```

The expression below

```
  (SQUARE-EVENS NIL)
```

should evaluate to

```
  NIL
```